A Graph-Based Conceptual Model for MILP Modeling

Juan Moreno(✉), Josefa Mula, Raúl Poler, and Jordi Almendros

Research Centre on Production Management and Engineering (CIGIP), Universitat Politècnica de València, Alcoy, Alicante, Spain
{jmoreno,fmula,rpoler,jalmendros}@cigip.upv.es

Abstract. Production and supply chain optimization problems have grown increasingly complex due to advancements in computing power and mixed-integer linear programming (MILP) solvers. Algebraic modeling languages (AMLs) such as Pyomo, JuMP, AMPL and GAMS, enable researchers and practitioners to describe these problems in a high-level, declarative fashion in a given programming language. This paper presents a comparative analysis of these AMLs with an emphasis on their use in real-world problems. A conceptual framework that empowers traditional modeling languages is presented as a potential graph-based MILP modeling platform. It enables the creation of sets, variables, constraints and objectives as nodes on graph. A visual equation editor is proposed to increase the visibility, explainability and the modeling process. The framework also provides a solver and AML agnostic interfaces by allowing users to seamlessly switch between different solving environments. The article concludes with a discussion of further research directions.

Keywords: Supply Chain · Optimization · Modeling · MILP

1 Introduction

Production and supply chain optimization problems are growing in scale and complexity, driven by advances in MILP solvers (Desrosiers & Lübbecke 2011; Clautiaux & Ljubić 2024) and increased computational power. Modern algorithms and hardware improvements have made it possible to tackle large-scale, multi-echelon supply chain models that were once computationally infeasible. MILP formulations now capture intricate real-world constraints. For instance, MILP models enable us to solve problems of all different disciplines, from supply chain (Díaz-Madroñero, Mula & Peidro 2014; Díaz-Madroñero, Peidro & Mula 2015; Díaz-Madroñero, Mula & Peidro 2017) to routing (Li et al. 2014; Madankumar & Rajendran 2018; Clautiaux & Ljubić 2024) and energy (Bozchalui et al. 2012; Gabrielli et al. 2018).

Algebraic modeling languages (AMLs) bridge the gap between the mathematical description of these complex problems and solver-ready formulations. In recent years, the open-source community has released different AMLs, most of which come as libraries of different programming languages like JuMP-Julia (Dunning, Huchette & Lubin 2017;

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
E. Barbadilla-Martín et al. (Eds.): CIO 2025, LNDECT 276, pp. 282–287, 2025.
https://doi.org/10.1007/978-3-032-10126-6_45

Lubin et al. 2023), PuLP-Python, and Pyomo-Python (Hart, Watson & Woodruff 2011). As these AMLs are part of general programming languages, they can be merged and used through other subsystems, which enhances the use of MILP formulations. Other AMLs are proprietary and have an economic cost; for example, AMPL (Fourer, Gay & Kernighan 1990) and GAMS. This paper presents a review and comparison of four widely used AMLs (Pyomo, JuMP, AMPL and GAMS) with a focus on MILP production and supply chain applications. It also proposes a conceptual framework that combines algebraic modeling with enhanced interactivity, visualization and documentation capabilities.

The remainder of the paper is structured as follows. Section 2 presents the research methodology. Section 3 describes the AMLs review. Section 4 proposes the conceptual framework. Section 5 includes the conclusions and proposes a road map for further research.

2 Methodology

The research scope of this article lies in evaluating and comparing existing AMLs by examining their syntax, usability and performance in representing MILP models to propose a conceptual framework for a new MILP modeling platform. The focus is on how these languages allow modelers to express standard optimization formulations (e.g. in a human-readable, declarative form) and to identify the key missing features that AMLs should have. The scope covers the review of Pyomo, JuMP, AMPL and GAMS.

To do so, we briefly describe what a MILP problem is and what the main characteristics are of current AMLS. MILP is an optimization problem in which some decision variables are constrained to be integers, while others can be continuous. It follows the standard linear programming form $Ax = b$, where A is a coefficient matrix ($m \times n$), x is the vector of decision variables and b is a constraint vector. The goal is to minimize (or maximize) a linear objective function $c^T x$ subject to linear equality and inequality constraints. So the main MILP components are the decision variables, the cost/profit and technological parameters, constraints and the objective function.

Mathematical languages allow us to express problems in a more readable way than merely defining the matrix using their own indices. We employ sets to group related elements like products, time periods or locations, which enables modelers to index variables, parameters and constraints for scalability and ease of modification. Additionally, summation operator $\sum$ can be employed to aggregate these indexed elements, where variables can be multiplied by the defined parameters. In this case, we require the multiplication of variables and the problem becomes non linear. Thus AMLs allow us to bridge the gap between the mathematical notation and the code submitted to the MILP solver. Even though MILP seems a simple modeling framework, it enables us to express real-world complex problems. To compare different AMLs, we first describe a simple assignment problem formulated as a MILP. We define two sets: I, set of agents with index i, and J, set of tasks with index j. Each agent must be assigned exactly one task, and *vice versa*. Therefore, we define binary variables X_{ij}, where $i \in I, j \in J$, to represent that i does task j. Furthermore, we define parameter c_{ij} to represent the cost of i doing

task j. The MILP problem is the following:

$$z = \sum_{i \in I} \sum_{j \in J} X_{ij} \cdot c_{ij} \tag{1}$$

$$\sum_{i \in I} X_{ij} = 1 \forall j \in J \tag{2}$$

$$\sum_{j \in J} X_{ij} = 1 \forall i \in I \tag{3}$$

Furthermore, we briefly discuss real-world use in MILP programs by analyzing the title and abstracts of the scientific literature.

3 AML Review

Pyomo is an open-source AML embedded in Python. It allows users to define models employing Python's rich data structures and object-oriented features. Pyomo supports a wide range of optimization problems and interfaces with many solvers (e.g., CPLEX, Gurobi, GLPK, CBC). Its expressiveness makes it well-suited for integrating optimization into larger Python-based data processing and analysis workflows. One noted challenge with Pyomo is that model generation can be slower for very large instances due to Python's interpreted nature. JuMP is an open-source AML built in Julia and designed for high performance. Its syntax closely resembles mathematical notation by using macros, such as @variable, @constraint and @objective, to allow concise model formulation. JuMP benefits from Julia's just-in-time (JIT) compilation by offering fast model generation, although there is a startup cost for JIT compilation. Its excellent integration with a wide range of solvers *via* the MathOptInterface makes JuMP a good option for large-scale production and supply chain models. AMPL is a well-established, proprietary AML that emphasizes clarity and conciseness. By separating model and data, AMPL allows users to define a general model before applying it to different datasets. It supports extensive solver compatibility and automatically extends indexed expressions, which make it highly efficient for large and sparse models. Despite its powerful features, AMPL's proprietary nature and standalone environment can make integration with other software more challenging. GAMS is designed for large-scale, complex models. It excels in handling multidimensional indices and sparse data through specialized syntax. GAMS provides extensive solver support and is highly efficient in model generation and instance creation. Table 1 describes how JuMP, AMPL, PyOMO and GAMS define the assignment problem MILP (1)–(3).

Each AML has its own strength. Some seek speed and performance, while others focus on easiness of use and interoperability with other systems. We can note that they all make the code as close to the mathematical notation as possible. Due to the use of plain text as a user code, blackboard experience in definition and modeling is not possible, which can sometimes lead to difficulties in understanding the relations among parameters and variables with constraints and the objective function.

Figure 1 analyzes the evolution of MILP scientific literature. Most of it focuses on energy optimization, scheduling and planning and control. Additionally, a growing trend in modeling and optimizing processes has been identified over the years.

Table 1. Example of the code description of each model's component

	JuMP	AMPL
Sets	`I = [...]` `J= [...]`	`set I;` `set J;`
Variables	`@var(m, x[i in I, j in J], Bin)`	`var x {i in I, j in J} binary;`
Parameter	`c = Dict((i,j) => cost_i_j for i in I, j in J)`	`param c {I, J};`

Constraint	`@const(m,[i in I], sum(x[i, j]  for j in J) == 1)`
	`@const(m, [j in J], sum(x[i, j] for i in I) == 1)`
		`subject to Agent {i in I}:`
		`  sum {j in J} x[i, j] = 1;`
		`subject to Task {j in J}:`
		`  sum {i in I} x[i, j] = 1;`

Objective function	`@obj(m, Min, sum(c[i, j] * x[i, j]` `  for i in I, j in J))`	`minimize TotalCost:` `sum {i in I, j in J} c[i, j] x[i,j];`
	Pyomo	**GAMS**

Sets	`m.I = Set()`
	`m.J = Set()`
		`Sets`
		`  i Agents / /,`
		`  j Tasks / /;`

Variables	`m.x = Var(m.I, m.J, domain=Binary)`	`Binary Variable x(i,j);`
Parameter	`m.c = Param(m.I, m.J)`	`Parameters c(i,j) Cost data;`

Constraint	`def agent_rule(m, i):`
	`  return sum(m.x[i, j] for j in m.J) == 1`
	`m.agent_assign = Constraint(m.I, rule=agent_rule)`
	`def task_rule(model, j):`
	`  return sum(m.x[i, j] for i in m.I) == 1`
	`m.task_assign = Constraint(m.J, rule=task_rule)`
		`Equations`
		`  assign_i(i) Assignment constraint`
		`  assign_j(j) Assignment constraint ;`
		`assign_i(i).. sum(j, x(i,j)) =e= 1;`
		`assign_j(j).. sum(i, x(i,j)) =e= 1;`

Objective function	`def obj_rule(m):`
	`  return sum(m.c[i, j] * m.x[i, j]`
	`  for i in m.I for j in m.J)`
	`m.obj = Objective(rule=obj_rule, sense=minimize)`
		`Positive Variable z;`
		`Equations`
		`  obj Objective function;`
		`obj.. z =e= sum((i,j),c(i,j)*x(i,j));`

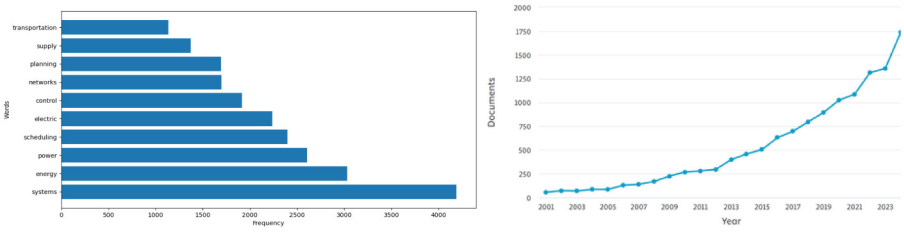

Fig. 1. On the left, evaluation of the frequency of related keywords; on the right, number distribution per year of Scopus search: (TITLE-ABS-KEY (milp OR "mixed-integer linear programming" OR "mixed integer programming") AND TITLE-ABS-KEY (optimi*ation)) AND PUBYEAR >2000 AND PUBYEAR <2026 AND (LIMIT-TO (DOCTYPE, "ar")) AND (LIMIT-TO (LANGUAGE, "English")).

4 Conceptual Framework

We propose a new conceptual framework for MILP modeling to tackle the main difficulties identified in Sect. 3. Our proposal is a visual tool to help to model MILP programs. It is done through a graph-based representation, where graph nodes represent sets, variables, parameters, constraints and the objective function. Each node is added to the graph by a form, where a visual equation editor is shown to enhance node visibility. The way to define the MILP by the user is done through the corresponding mathematical equations

without an intermediate programming code. Once a node is defined, it is linked with the other nodes through edges by the following rule: variables and parameters are related to a constraint and/or objective function when used in them. Furthermore, a drawing canvas is provided to visualize the model. Once the model is defined, we obtain the following outputs: (i) the code in the selected AML to solve the MILP; (ii) the nomenclature and mathematical notation of the MILP model in a latex and as html, .doc, or .pdf files, among others, for dissemination purposes. Hence our conceptual proposal is to use the existing AMLs as an interface between our MILP modeling tool and solvers. Hence the assignment problem (1)–(3) would have the following associated graph within the proposed framework (Fig. 2).

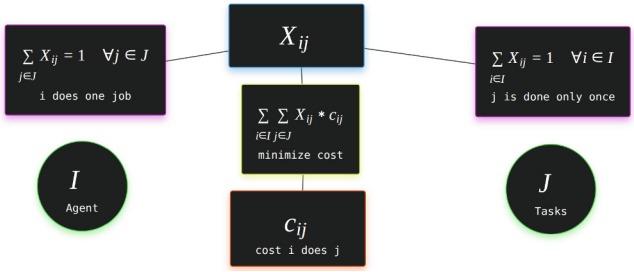

Fig. 2. Graph-based representation of the assignment problem

5 Conclusions

The analysis of AMLs (Pyomo, JuMP, AMPL and GAMS) reveals that although it effectively translates MILP formulations into solver-ready codes, they sometimes lack a visual representation of the relations among sets, decision variables, parameters, constraints and the objective function. To address this gap, we propose a novel MILP conceptual framework that empowers visualization and interactivity by representing each model component as a node in a graph. Regarding theoretical implications, this conceptual framework bridges the gap between mathematical abstraction and computational implementation and provides solver and AMLs interfaces to switch easily between different solving and AMLs environments to contribute to more robust and user-friendly MILP modeling experiences. As regards managerial implications, the proposed framework synchronizes the generated MILP with the scientific documentation by eliminating errors that are made in the translation phase of the mathematical equations to the AML.

A forthcoming work aims to validate the proposed conceptual framework through the development of the MILP modeling tool by testing with researchers and practitioners to standardize the notation and to identify future features. Finally, another aim is to release the tool as an open-source platform to integrate future AMLs and extend notation capabilities.

Acknowledgments. This study was funded by the Horizon Europe Framework Programme (GA No. 101177842) "Unified Modeling and Automated Scheduling for Manufacturing as a Service" (UniMaaS).

References

Bozchalui, M.C., Hashmi, S.A., Hassen, H., Cañizares, C.A., Bhattacharya, K.: Optimal operation of residential energy hubs in smart grids. IEEE Trans. Smart Grid. **3**(4), 1755–1766 (2012)

Clautiaux, F., Ljubić, I.: Last fifty years of integer linear programming: a focus on recent practical advances. Eur. J. Oper. Res. **324**, 707 (2024)

Desrosiers, J., Lübbecke, M.E.: Branch-Price-and-cut algorithms. In: Wiley Encyclopedia of Operations Research and Management Science. John Wiley & Sons, Ltd (2011)

Díaz-Madroñero, M., Mula, J., Peidro, D.: A review of discrete-time optimization models for tactical production planning. Int. J. Prod. Res. **52**(17), 5171–5205 (2014)

Díaz-Madroñero, M., Mula, J., Peidro, D.: A mathematical programming model for integrating production and procurement transport decisions. Appl. Math. Model. **52**, 527–543 (2017)

Díaz-Madroñero, M., Peidro, D., Mula, J.: A review of tactical optimization models for integrated production and transport routing planning decisions. Comput. Ind. Eng. **88**, 518–535 (2015)

Dunning, I., Huchette, J., Lubin, M.: JuMP: a modeling language for mathematical optimization. SIAM Rev. **59**(2), 295–320 (2017)

Fourer, R., Gay, D.M., Kernighan, B.W.: A modeling language for mathematical programming. Manag. Sci. **36**, 519 (1990)

Gabrielli, P., Gazzani, M., Martelli, E., Mazzotti, M.: Optimal design of multi-energy systems with seasonal storage. Appl. Energy. **219**, 408–424 (2018)

Hart, W.E., Watson, J.P., Woodruff, D.L.: Pyomo: modeling and solving mathematical programs in python. Math. Program. Comput. **3**(3), 219–260 (2011)

Li, B., Krushinsky, D., Reijers, H.A., Van Woensel, T.: The share-A-ride problem: people and parcels sharing taxis. Eur. J. Oper. Res. **238**(1), 31–40 (2014)

Lubin, M., Dowson, O., Garcia, J.D., Huchette, J., Legat, B., Vielma, J.P.: JuMP 1.0: recent improvements to a modeling language for mathematical optimization. Math. Program. Comput. **15**(3), 581–589 (2023)

Madankumar, S., Rajendran, C.: Mathematical models for green vehicle routing problems with pickup and delivery: a case of semiconductor supply chain. Comput. Oper. Res. **89**, 183–192 (2018)